

SONGO

Version 1, Jeu local (un seul fichier HTML)

Réalisé par :

AYEMTSA DJOUDA JORAN FRED-24F2665

À propos de cette version

La version 1 est un fichier HTML entièrement autonome (songo.html). Aucune dépendance externe, aucun serveur requis. Deux joueurs partagent le même écran et le même clavier/souris.

1. Fonctionnement du jeu

1.1 Présentation

Le Songo est un jeu de stratégie à deux joueurs de la famille des jeux de semailles. Il est pratiqué par les peuples Ekang d'Afrique centrale (Eton, Bulu, Ewondo, Bété). Le but est de capturer plus de 40 graines sur les 70 en jeu.

1.2 Mise en place

Le plateau comporte 2 rangées de 7 cases. Chaque case contient 5 graines au départ (70 graines au total). Le Joueur Sud joue la rangée du bas, le Joueur Nord la rangée du haut.

Case	N1	N2	N3	N4	N5	N6	N7
Nord (rangée 0)	5	5	5	5	5	5	5
Sud (rangée 1)	5	5	5	5	5	5	5

État initial du plateau

1.3 Déroulement d'un tour

Le joueur actif choisit une case de son camp (cases bleues surlignées). Il ramasse toutes ses graines et les distribue une à une en boucle :

- Son camp : de droite à gauche (case 7 → case 1).
- Camp adverse : de gauche à droite (case 1 → case 7).
- Si la case choisie contient plus de 13 graines : tour complet, la case de départ est sautée.

1.4 Récoltes (prises)

Uniquement dans le camp adverse. La dernière graine distribuée doit tomber dans une case contenant 1–3 graines (total 2–4 après dépôt) → toutes capturées. Prise à la chaîne : on remonte les cases précédentes contenant 2–4 graines.

- Case n°1 adverse : prise d'1 seule graine, uniquement si ≥ 14 graines distribuées.

1.5 Règle de solidarité

Si le camp adverse est vide : le joueur doit distribuer au moins 7 graines dans ce camp. Sinon, il distribue le maximum possible. Si impossible, la partie s'arrête.

1.6 Interdits

- Interdit de semer 1 ou 2 graines chez l'adversaire via sa case 7.
- Interdit de vider complètement le camp adverse lors d'une prise.

1.7 Fin de partie

Condition	Résolution
Un joueur atteint 40 graines	Ce joueur gagne
Moins de 10 graines sur le plateau	Chaque joueur récupère son camp ; le plus riche gagne
Solidarité impossible	Idem
Égalité	Partie nulle

2. Principe

2.1 Nature du jeu

Le Songo est un jeu combinatoire parfait à information complète : les deux joueurs voient l'intégralité du plateau. Il n'y a aucun hasard. La victoire repose sur la planification et l'anticipation du circuit de distribution.

2.2 Tension stratégique

- Capturer un maximum de graines dans le camp adverse.
- Éviter de laisser des cases vulnérables (2–3 graines) dans son propre camp.
- Gérer la solidarité : parfois sacrifier une capture pour alimenter l'adversaire.

2.3 Circuit de distribution

La boucle de distribution détermine où tombe la dernière graine — c'est ce point d'arrivée qui conditionne une prise. Anticiper ce point pour chaque case jouable est le cœur du calcul stratégique.

Interaction locale — deux joueurs, un écran

Dans cette version, les deux joueurs se passent le clavier/la souris. Le plateau est affiché de manière symétrique : Nord en haut, Sud en bas. Seules les cases du joueur actif sont cliquables (surlignées en bleu).

3. Algorithmme

3.1 Représentation des données

```
board = [ [5, 5, 5, 5, 5, 5, 5], // rangée 0 : Nord [5, 5, 5, 5, 5, 5, 5] //  
rangée 1 : Sud ] scores = [0, 0] // [scoreNord, scoreSud] currentPlayer = 1 // 1  
= Sud, 2 = Nord gameOver = false
```

3.2 Navigation — nextPos(row, col, player)

Calcule la case suivante dans le sens de distribution. Le circuit est asymétrique selon le joueur (Sud tourne dans un sens, Nord dans l'autre) :

```
// Joueur Sud : row1 col- → row0 col+ // Joueur Nord : row0 col+ → row1 col-  
function nextPos(row, col, player) { if (player === 1) { if (row === 1) return  
col > 0 ? [1, col-1] : [0, 0]; else return col < 6 ? [0, col+1] : [1, 6]; } else  
{ if (row === 0) return col < 6 ? [0, col+1] : [1, 6]; else return col > 0 ? [1,  
col-1] : [0, 0]; } }
```

3.3 Distribution des graines

```
seeds = board[own][col]; board[own][col] = 0 row = own; c = col; s = seeds while  
(s > 0) { [row, c] = nextPos(row, c, player) if (row==own && c==col && seeds>13)  
{ s--; continue } // sauter case départ board[row][c]++; s-- } // (row, c) =  
position de la dernière graine
```

3.4 Calcul des prises

```
if (lastRow === oppRow) { if (lastCol === oppFirstCol && seeds >= 14) { capturer  
1 graine // case n°1 : règle spéciale } else { while (pos dans camp adverse &&  
cnt in [2..4] && pos != case_1) { capturer cnt; board[pos] = 0; pos =  
prevPos(pos) } } }
```

3.5 Vérification des interdictions

```
// Interdit : vider le camp adverse if (board[opp].every(x => x === 0) &&  
captureList.length > 0) { for ([r,c,cnt] of captureList) board[r][c] += cnt //  
annuler captureList = [] }
```

3.6 Cases jouables — solidarité

```
function getSelectableCells(board, player) { if (!oppEmpty) return allOwnCells  
// cas normal // Solidarité : filtrer les coups atteignant >=7 graines chez  
l'adversaire const reach7 = ownCells.filter(c => distInOpp(c) >= 7) if  
(reach7.length > 0) return reach7 // Sinon : coup qui distribue le maximum  
maxDist = max(distInOpp(c) for c in ownCells) return ownCells.filter(c =>  
distInOpp(c) === maxDist) }
```

3.7 Fin de partie — checkEnd()

Condition vérifiée	Action
scores[i] >= 40	Fin immédiate — joueur i gagne
totalOnBoard() < 10	Chaque joueur récupère son camp, fin
Camp adverse vide + solidarité impossible	Idem

4. Organisation du code

4.1 Structure du fichier unique (songo.html)

Tout le jeu tient dans un seul fichier HTML. Le code JavaScript est organisé en 5 blocs fonctionnels séparés par des commentaires clairs :

Bloc	Rôle	Fonctions / Variables
État	Variables globales du jeu	board, scores, currentPlayer, gameOver
Utilitaires navigation	Déplacement sur le plateau	ownRow(), oppRow(), nextPos(), prevPos(), oppFirstCol()
Règles	Cases jouables, solidarité	getSelectableCells(), simulateDistInOpp()
Moteur	Exécution coup, prises, fin	playMove(), checkEnd(), endGame()
Rendu UI	Mise à jour DOM	render(), updateStatus(), addLog(), clearLog(), initGame()

4.2 Flux d'exécution complet

```
initGame() ■■ board = [[5×7],[5×7]]; scores=[0,0]; currentPlayer=1 ■■ render()
[Joueur clique une case] ■■ playMove(col) ■■ getSelectableCells() → valider le
coup ■■ Distribution : boucle nextPos() jusqu'à épuisement ■■ Calcul prises +
prise à la chaîne (prevPos) ■■ Vérif interdit (vider camp adverse) ■■
scores[player] += totalCaptured ■■ checkEnd() → si fin : endGame() + afficher
résultat ■■ currentPlayer = (player === 1) ? 2 : 1 ■■ render(highlight) +
updateStatus()
```

4.3 Séparation logique / affichage

La logique (playMove, checkEnd, getSelectableCells) opère uniquement sur les tableaux board et scores, sans jamais toucher le DOM. La fonction render() est l'unique point d'entrée pour la mise à jour visuelle. Elle reconstruit les deux rangées entièrement à chaque appel, évitant toute désynchronisation.

4.4 Mise en évidence visuelle (highlight)

```
// Objet transmis à render() après chaque coup highlight = { last: [row, col],
// Dernière case distribuée → .last-drop (vert) captured: [[r,c], [r,c], ...]
// Cases capturées → .captured (rouge) } // Les cases jouables sont toujours
.selectable (bleu)
```

Récapitulatif — Version 1

- **1 fichier HTML** autonome, zéro dépendance, zéro serveur.
- **2 joueurs** sur le même écran (tour par tour).
- **5 blocs JS** : état, navigation, règles, moteur, rendu.
- **Logique pure** séparée du DOM — render() reconstruit à chaque coup.
- **Toutes les règles** implémentées : solidarité, interdits, prises à la chaîne.